



Chess LSPI Agent

https://github.com/NikolaosSopiadis/Chess_LSPI

1 Παρουσίαση προβλήματος

Το project ξεκινάει με την υλοποίηση του πυρήνα σκακιού (chess engine core), το οποίο βρίσκεται στο φάκελο "chess_core". Το συγκεκριμένο τμήμα είναι υπεύθυνο για την αναπαράσταση της κατάστασης του παιχνιδιού και την τήρηση όλων των κανόνων του σκακιού. Η εκπαίδευση LSPI βασίζεται στο παρόν τμήμα.

2 Περιβάλλον: Πυρήνας σκακιού

Πριν μπορέσουμε να ασχοληθούμε με τον πράκτορα, χρειαζόμαστε ένα κατάλληλο περιβάλλον. Ακολουθούν τα μέρη που αποτελούν τον πυρήνα. Ο σχετικός κώδικας βρίσκεται στον φάκελο "chess_core". Η κύρια λειτουργικότητα βρίσκεται στην κλάση "Board" του αρχείου "board.py"

2.1 Αναπαράσταση σκακιέρας

Η σκακιέρα αναπαρίσταται εσωτερικά ως ένα array 64 τετραγώνων μεγέθους ενός byte: "board: bytearray". Κάθε τετράγωνο περιέχει ένα πιόνι κωδικοποιημένο ως ακέραιο αριθμό.

Ο υπολογισμός της σειράς (rank) και στήλης (file) κάθε πιονιού γίνεται αριθμητικά με τις λογικές ακόλουθες πράξεις μεταξύ bits:

```
1 rank = square >> 3 # equivalent to square / 8  
2 file = square & 7 # equivalent to square % 8
```

Επιπλέον χρησιμοποιούνται οι παρακάτω μεταβλητές για την διατήρηση της κατάστασης του παιχνιδιού και την τήρηση των κανόνων:

- Παίκτης που παίζει: boolean μεταβλητή "is_white_to_move: bool"
- Δικαιώματα ροκέ (castling): ακέραια μεταβλητή "castling_rights: int", όπου τα 4 πρώτα bits αντιστοιχούν στο δικαίωμα ροκέ για:
 - Λευκά στη μεριά του βασιλιά
 - Λευκά στη μεριά της βασίλισσας
 - Μαύρα στη μεριά του βασιλιά
 - Μαύρα στη μεριά της βασίλισσας
- Στόχος en-passant: "en_passant_target: int | None"
- Ρολόι ημικίνησης: "halfmove_clock: int", ημικίνηση αποκαλείται αφού παίξει ο κάθε παίκτης
- Αριθμός κινήσεων: "fullmove_number: int", κίνηση θεωρείται αφού έχουν παίξει και οι δυο παίκτες.
- Θέση λευκού βασιλιά: "white_king_sq: int"
- Θέση μαύρου βασιλιά: "black_king_sq: int"
- Zobrist key: "zkey: int", χρησιμοποιείται για την αναπαράσταση κάθε μοναδικής θέσης

- Πίνακας επαναλήψεων της κάθε θέσης: “rep_counts: dict[int, int]”, hash map που επιστρέφει τον αριθμό εμφανίσεων της θέσης, παίρνει ως κλειδί το zobrist key. `rep_counts[zobrist_key] = number_of_times_this_position_has_occurred`
- Πίνακας των κινήσεων: “rep_stack: list[int]”, κάθε κίνηση που παίζεται αποθηκεύεται εδώ.

```

1 @dataclass(frozen=True, slots=True)
2 class BoardState:
3     board: bytes
4     is_white_to_move: bool
5     castling_rights: int
6     en_passant_target: int | None
7     halfmove_clock: int
8     fullmove_number: int
9     white_king_sq: int
10    black_king_sq: int
11    zkey: int
12    rep_counts: tuple[tuple[int, int], ...]
13    rep_stack: tuple[int, ...]

```

2.2 Κωδικοποίηση πιονιών

Ο σχετικός κώδικας βρίσκεται στο αρχείο “chess_core/piece.py”. Τα πιόνια κωδικοποιούνται ως ένας αριθμός 4 bit, 1 bit για το χρώμα και 3 bit για το είδος. Ακολουθούν οι αριθμητικές αναπαραστάσεις τους:

- WHITE = 0 (0|xxx)
- BLACK = 8 (1|xxx)
- PAWN = 1 (x|001)
- KNIGHT = 2 (x|010)
- BISHOP = 3 (x|011)
- ROOK = 4 (x|100)
- QUEEN = 5 (x|101)
- KING = 6 (x|110)

Ακολουθούν μερικά παραδείγματα:

- WHITE PAWN = 1 (0|001)
- BLACK PAWN = 9 (1|001)
- WHITE KING = 6 (0|110)
- BLACK KING = 14 (1|110)

Ο έλεγχος πιονιών γίνεται με τη χρήση βοηθητικών συναρτήσεων όπως οι παρακάτω:

```

1 p.piece_type(piece)
2 p.is_white(piece)
3 p.is_black(piece)

```

2.3 Αναπαράσταση κινήσεων

Οι κινήσεις αναπαριστώνται από ένα αντικείμενο “Move”, με τα ακόλουθα πεδία:

```

1 @dataclass(frozen=True, slots=True)
2 class Move:
3     src_square: int # idx of source square
4     dst_square: int # idx of destination square
5
6     flags: int      = F_QUIET
7     promotion: int = PROMO_NONE

```

```

8     captured_piece: int = 0
9
10    # flag bits (plain ints)
11    F_QUIET      = 0
12    F_CAPTURE    = 1 << 0
13    F_EN_PASSANT = 1 << 1
14    F_CASTLE     = 1 << 2
15    F_PROMOTION  = 1 << 3
16    F_DOUBLE_PAWN = 1 << 4

```

Κάθε κίνηση περιλαμβάνει πάντα τα τετράγωνα αφετηρίας και προορισμού. Επιπλέον μπορεί να περιλαμβάνει πληροφορία για το είδος της κίνησης, όπως αν έγινε κάποια αρπαγή, προαγωγή, διπλή κίνηση απλού πιονιού, ροκέ (castle), ή en passant. Η πληροφορία που αποθηκεύεται είναι αρκετή για να μπορεί να γίνει αναίρεση των κινήσεων, κάτι απαραίτητο για την εύρεση νόμιμων κινήσεων καθώς και για αναζήτηση κινήσεων.

2.4 Γεννήτρια ψευδο-νόμιμων κινήσεων

Το πρώτο βήμα για τη γεννήτρια κινήσεων, είναι η εύρεση ψευδο-νόμιμων (pseudo-legal) κινήσεων, δηλαδή κινήσεων που τηρούν τους κανόνες κίνησης του πιονιού, αλλά ενδέχεται να αφήνουν τον βασιλιά σε σαχ (check). Παράδειγμα μιας τέτοιας κίνησης είναι η μετακίνηση ενός ίππου, ο οποίος είναι καρφωμένος στον βασιλιά από αξιωματικό του αντιπάλου. Αν μετακινηθεί, επιτρέπει στον αντίπαλο αξιωματικό να φάει τον βασιλιά.

Το chess engine δημιουργεί τις ακόλουθες ψευδο-νόμιμες κινήσεις για κάθε πiónι:

- Pawn moves:
 - single pushes
 - double pushes from starting rank
 - captures
 - promotions
 - en-passant
- Knight moves:
 - fixed L-shaped offsets
- Bishop moves:
 - sliding diagonals
- Rook moves:
 - sliding ranks and files
- Queen moves:
 - rook-like + bishop-like sliding
- King moves:
 - one-square movement
 - castling candidates

2.5 Γεννήτρια νόμιμων κινήσεων

Η γεννήτρια νόμιμων κινήσεων απορρίπτει παράνομες ψευδο-νόμιμες κινήσεις, παίζοντας κάθε υποψήφια κίνηση και ελέγχοντας αν ο βασιλιάς παραμένει ασφαλής.

```

1 legal_moves = []
2
3 for move in pseudolegal_moves:
4     make move temporarily
5
6     if own king is not in check:
7         legal_moves.append(move)
8
9     undo move

```

Η παραπάνω υλοποίηση είναι ένας απλός και αξιόπιστος έλεγχος νομιμότητας. Δεν είναι ο πιο γρήγορος αλλά είναι ικανοποιητικός για το παρόν project.

2.6 Έλεγχος σαχ (check)

Ο έλεγχος σαχ (check), δηλαδή αν ο βασιλιάς απειλείται, γίνεται με την αποθήκευση της θέσης των 2 βασιλιάδων. Έτσι αποφεύγεται η αναζήτηση ολόκληρης της σκακιέρας κάθε φορά που θέλουμε να ελέγξουμε αν ο βασιλιάς βρίσκεται υπό επίθεση.

Ο πυρήνας (chess core) παρέχει μεθόδους όπως

```
1 king_square(self, white: bool) -> int
2 in_check(self, white: bool) -> bool
3 is_square_attacked(self, square: int, by_white: bool) -> bool
```

2.7 Εκτέλεση και αναίρεση κινήσεων

Ο πυρήνας (chess core) υποστηρίζει 2 είδη κινήσεων: μόνιμη και προσωρινή.

Η μόνιμη κίνηση εκτελείται κατά τη διάρκεια του παιχνιδιού με την εντολή “board.make_move(move)”.

Η προσωρινή εκτέλεση κίνησης χρησιμοποιείται κατά για σκοπούς γέννησης κινήσεων (move generation), αναζήτησης και παραγωγής συναρτήσεων βάσης (feature extraction) για το LSPI.

```
1 @contextmanager
2 def temporary_move(self, move: Move) -> Iterator[None]:
3     """
4     Temporarily make a move and automatically undo it.
5
6     Usage:
7         with board.temporary_move(move):
8             ...
9     """
10    undo = self._do_move(move)
11    try:
12        yield
13    finally:
14        self._undo_move(undo)
```

Το αντικείμενο undo πρέπει να μπορεί να αποκαταστήσει πλήρως την κατάσταση της σκακιέρας. Για αυτό, αποθηκεύει τις προηγούμενες τιμές της κατάστασης της σκακιέρας (BoardState), την κίνηση που παίχτηκε, καθώς και μερικά επιπλέον βοηθητικά πεδία όπως το πiónι που μετακινήθηκε, το πiónι που αρπάχτηκε σε περίπτωση αρπαγής (capture) και την κίνηση του πύργου σε περίπτωση ροκέ (castling).

```
1 @dataclass(slots=True)
2 class Undo:
3     move: Move
4     moved_piece: int
5     captured_piece: int
6     captured_square: int          # usually dst; for en passant it's behind dst
7     prev_castling_rights: int
8     prev_en_passant_target: int | None
9     prev_halfmove_clock: int
10    prev_is_white_to_move: bool
11    prev_white_king_sq: int
12    prev_black_king_sq: int
13    prev_zkey: int
14    rook_src: int = -1             # for castling
15    rook_dst: int = -1
```

2.8 Zobrist hashing

Το Zobrist hash είναι ένα hash που αποθηκεύει ολόκληρη τη θέση της σκακιέρας. Έτσι, αντί να συγκρίνει απευθείας ολόκληρες καταστάσεις σκακιέρας - μια δομή με πολλά πεδία, επομένως κάτι υπολογιστικά ακριβό - ο πυρήνας χρησιμοποιεί ένα αριθμητικό κλειδί το οποίο ταυτοποιεί την τωρινή θέση με πολύ μεγάλη πιθανότητα.

Το Zobrist key συνήθως περιλαμβάνει

- Τα πιόνια σε κάθε τετράγωνο της σκακιέρας
- Την πλευρά που παίζει
- Δικαιώματα ροκέ (castling rights)
- Τετράγωνο στο οποίο μπορεί να γίνει κίνηση en passant

Έτσι, 2 θέσεις με πιόνια στις ίδες θέσεις αλλά διαφορετική πλευρά που παίζει, ή με διαφορετικά δικαιώματα ροκέ, παράγουν διαφορετικά κλειδιά.

Η ιδέα είναι η εξ αρχής δημιουργία τυχαίων αριθμών για κάθε πιθανό στοιχείο κατάστασης. Τότε το hash κατασκευάζεται κάνοντας XOR τους σχετικούς τυχαίους αριθμούς.

```

1 zkey = 0
2
3 for square in squares:
4     piece = board[square]
5     if piece != EMPTY:
6         zkey ^= ZOBRIST_PIECE[piece][square]
7
8 if black_to_move:
9     zkey ^= ZOBRIST_SIDE_TO_MOVE
10
11 zkey ^= ZOBRIST_CASTLING[castling_rights]
12
13 if en_passant_target is not None:
14     zkey ^= ZOBRIST_EP[file_of(en_passant_target)]

```

The advantage is that the key can be updated incrementally when a move is made. For example, moving a knight from g1 to f3 can be handled by XOR-ing out the knight on g1 and XOR-ing in the knight on f3. Το κλειδί μπορεί να ενημερώνεται κάθε σταδιακά όταν γίνεται μια κίνηση. Για παράδειγμα, η κίνηση ενός ίππου από το g1 στο f3 μπορεί να αποτυπωθεί με τα εξής βήματα:

- XOR τον ίππο στο g1 - υπήρχε από πριν στο κλειδί, οπότε τώρα αφαιρείται.
- XOR τον ίππο στο f3 - προστίθεται στο κλειδί.

Υπενθύμιση: Ο τελεστής XOR έχει την εξής ιδιότητα: $a \oplus b \oplus b = a$

```

1 zkey ^= ZOBRIST_PIECE[WHITE_KNIGHT][g1]
2 zkey ^= ZOBRIST_PIECE[WHITE_KNIGHT][f3]

```

2.9 Επανάληψη κινήσεων

Η επανάληψη της ίδιας θέσης 3 φορές οδηγεί σε ισοπαλία. Επομένως, είναι σημαντικό να μπορεί να γίνεται παρακολούθηση του αριθμού εμφάνισης της κάθε θέσης.

Μια αποδοτική υλοποίηση του παραπάνω είναι η αποθήκευση ιστορικού κλειδιών Zobrist (Zobrist key history). Όποτε εκτελείται μια κίνηση, το τωρινό Zobrist key προστίθεται στο ιστορικό επαναλήψεων. Όποτε αναιρείται μια κίνηση, το κλειδί πρέπει να αφαιρείται από το ιστορικό.

```

1 if move:
2     self._repetition[self._zkey] += 1
3 if undo:
4     self._repetition[self._zkey] -= 1

```

Ο έλεγχος τριπλής επανάληψης μπορεί να γίνει εύκολα και γρήγορα ελέγχοντας αν η θέση έχει εμφανιστεί 3 φορές στο ιστορικό:

```

1 self._repetition[self._zkey] >= 3

```

2.10 Εντοπισμός τερματικών καταστάσεων

The board provides a method like: Ο πυρήνας (chess core) παρέχει την ακόλουθη μέθοδο, η οποία επιστρέφει αν το παιχνίδι έχει τελειώσει και τον λόγο λήξης. Οι υποστηριζόμενοι λόγοι τερματισμού της παρτίδας είναι οι εξής:

- Ρουά ματ (checkmate): Νίκησε ο ένας παίκτης.

- Πατ (stalemate): Ισοπαλία λόγω έλλειψης κινήσεων.
- Τριπλή επανάληψη (threefold repetition): Ισοπαλία λόγω επανάληψης της ίδιας θέσης 3 φορές.
- Κανόνας 50 κινήσεων (50 move rule): Ισοπαλία μετά από συνεχόμενες 50 κινήσεις χωρίς καμία κίνηση απλού πιονιού (pawn) ή αρπαγής.
- Μη επαρκές υλικό (insufficient material): Ισοπαλία λόγω μη επαρκούς υλικού για ματ.

```

1 def game_end_state(self) -> tuple[bool, str]:
2     """
3     (done, reason) where reason in {
4     'checkmate', 'stalemate', 'threefold repetition', 'fifty-move rule',
5     'insufficient material', 'playing'
6     }.
7     """
8     moves = self.get_all_legal_moves()
9
10    # No legal moves -> mate or stalemate (these must override everything)
11    if not moves:
12        side = self._is_white_to_move
13        if self.in_check(side):
14            return True, "checkmate"
15        return True, "stalemate"
16
17    # Draws that can happen while moves still exist
18    if self.is_insufficient_material():
19        return True, "insufficient material"
20
21    if self._halfmove_clock >= 100:
22        return True, "fifty-move rule"
23
24    if self.is_threefold_repetition():
25        return True, "threefold repetition"
26
27    return False, "playing"

```

2.11 FEN

Ο πυρήνας υποστηρίζει την εισαγωγή και εξαγωγή θέσεων (positions) στο ταμπλό με μορφή FEN (Forsyth-Edwards Notation), δηλαδή με τη σύνηθη σκακιστική γραφή.

```

1 board.init_board(fen)
2 board.to_fen()

```

Το παραπάνω είναι πολύ σημαντικό για το project καθώς τα αλφαριθμητικά FEN χρησιμοποιούνται για:

- Αποθήκευση θέσεων (positions) σε συνθετικά dataset
- Αξιολόγηση πρακτόρων βάσει επιλεγμένων θέσεων (συγκεκριμένα ανοίγματα και κλεισίματα)
- Αποθήκευση της επόμενης κατάστασης των samples σε μορφή FEN για εκπαίδευση
- Debug σε συγκεκριμένες προβληματικές θέσεις

Στο LSPI dataset - το οποίο δημιουργείται ως ένα αρχείο JSON, κάθε sample μπορεί να αποθηκεύσει την επόμενη θέση ως FEN:

```

1 {
2     "phi": [...],
3     "r": 0.0,
4     "done": false,
5     "fen_next": "...
6 }

```

Έπειτα η διαδικασία εκπαίδευσης μπορεί να επανακασκευάσει την επόμενη κατάσταση και να αξιολογήσει την δράση (action) της πολιτικής του πράκτορα. Επομένως, αποτελεί έναν από τους συνδετικούς κρίκους μεταξύ περιβάλλοντος και μηχανικής μάθησης.

2.12 Έλεγχοι ορθότητας

Για την επιβεβαίωση της ορθής λειτουργίας του πυρήνα, κατά τη διάρκεια ανάπτυξης του κώδικα, γινόντουσαν συγκεκριμένοι έλεγχοι ορθότητας. Οι κώδικες δοκιμής βρίσκονται στον φάκελο “tests”. Δεν έχουν ενημερωθεί όλα τα test οπότε ορισμένα test ενδέχεται να μην τρέχουν με τις νέες αλλαγές στον κώδικα.

- PERFT (performance test, move path enumeration): Αποτελεί έλεγχο της γεννήτριας κινήσεων. Ελέγχει τον αριθμό των κινήσεων κάθε επιπέδου του δέντρου της γεννήτριας και τον συγκρίνει με προϋπολογισμένα αποτελέσματα. Έτσι επιβεβαιώνεται η μέτρηση όλων των νόμιμων κινήσεων, καθώς και η απόδοση της γεννήτριας.
- Zobrist: Έγινε έλεγχος της ορθής λειτουργίας του Zobrist hash.
- Board: Έγινε έλεγχος της ορθής λειτουργίας εκτέλεσης και αναίρεσης κινήσεων (move - undo).
- Insufficient material: Έγινε έλεγχος του ορθού τερματισμού του παιχνιδιού σε περίπτωση με επαρκούς υλικού.
- Piece: Έγινε έλεγχος της ορθής λειτουργίας του κώδικα αναπαράστασης των πιονιών.

2.13 Feature-extraction APIs

Με την εξέλιξη του πράκτορα, υπήρξε η ανάγκη για τη δημιουργία επιπλέον βοηθητικών APIs για την εξαγωγή features, προκειμένου να χρησιμοποιηθούν στο LSPI ως διανύσματα βάσης για την απεικόνιση πιο περίπλοκης συμπεριφοράς.

Μερικά αξιοσημείωτα παραδείγματα είναι τα παρακάτω:

```
1 piece_at(sq)
2 king_square(white)
3 queen_square(white)
4 move_gives_check(move)
5 move_gives_checkmate(move)
6 legal_moves_attacking_square(...)
```

2.14 GUI

Η γραφική διεπαφή (GUI) χρησιμοποιεί αρχιτεκτονική MVC (Model View Controller). Έτσι, ο πυρήνας μπορεί να δουλέψει τόσο με την παρουσία γραφικής διεπαφής όταν παίζει ο χρήστης, τόσο και χωρίς την παρουσία γραφικών όταν γίνεται εκπαίδευση.

Το model αποτελείται από τον πυρήνα (chess core) που περιγράφηκε παραπάνω, ο οποίος είναι υπεύθυνος όσα αναφέρθηκαν μέχρι τώρα.

Το view είναι υπεύθυνο για την απεικόνιση του ταμπλό και την εισαγωγή κινήσεων δια μέσω της γραφικής διεπαφής χρησιμοποιώντας το ποντίκι. Επιπλέον δίνει την επιλογή στο χρήστη να επιλέξει το μοντέλο του πράκτορα που θα χρησιμοποιηθεί από τον κάθε παίκτη (λευκά, μαύρα). Είναι σημαντικό να αναφερθεί ότι δεν έχει κανένα κομμάτι λογικής, αλλά δρα μόνο ως IO (Input Output). Για παράδειγμα, δεν αποφασίζει ανα μια κίνηση είναι νόμιμη ή όχι, απλά την δίνει στο μοντέλο, και το μοντέλο αποφασίζει τη νομιμότητα της. Σε περίπτωση παράνομης κίνησης, δεν κάνει τίποτα και περιμένει να δοθεί καινούρια.

Ο controller συνδέει το model και το view. Λέχεται είσοδο από τον χρήστη μέσω του controller και καλή την κατάλληλη συνάρτηση του model για να ενημερώσει την κατάσταση. Για παράδειγμα, όταν ο χρήστης μετακινήσει ένα πiónι, ο controller το μεταφράζει σε κίνηση και καλεί τη συνάρτηση του model *make_move(move)*. Επιπλέον, είναι υπεύθυνος για τη διεύθυνση ορισμένων λειτουργιών του view όπως η προβολή του ιστορικού κινήσεων, ή η αλλαγή του είδους παίκτη.

3 LSPI

Για την ενισχυτική μάθηση χρησιμοποιούμε τη μέθοδο LSPI (Least-Squares Policy Iteration), με στόχο την εκμάθηση μιας συνάρτησης αξιολόγησης σκακιού (chess evaluation function) από δεδομένα και ύστερα να χρησιμοποιήσουμε αυτή τη συνάρτηση αξιολόγησης για την επιλογή κίνησης.

Χάρης στη χρήση LSPI, ο πράκτορας μαθαίνει ένα διάνυσμα βαρών w πάνω σε ένα σύνολο από χαρακτηριστικών βάσεων - τα οποία από εδώ και στο εξής θα αναφέρονται ως *features*. Δοθείσης μιας θέσης, ο εξαγωγέας χαρακτηριστικών (features extractor) τη μετατρέπει σε ένα διάνυσμα από features $\phi(s)$, όπου s είναι η κατάσταση.

Ο πράκτορας αξιολογεί τη θέση του ταμπλό (position) χρησιμοποιώντας μια γραμμική συνάρτηση: $V(s) = w^\top \phi(s)$, όπου w το διάνυσμα βαρών και $\phi(s)$ το διάνυσμα χαρακτηριστικών (features).

Στο παρόν project, το μοντέλο του LSPI χρησιμοποιείται ως εκτιμήτρια μετα-κατάστασης (afterstate evaluator). Με τον όρο μετα-κατάσταση, ο οποίος θα αναφέρεται στο εξής ως *afterstate*, εννοείται η κατάσταση του παιχνιδιού που προκύπτει αμέσως μετά την εκτέλεση μιας κίνησης. Αυτό σημαίνει ότι ο πράκτορας εκτιμάει τη θέση που προκύπτει αφού εκτελέσει μια υποψήφια κίνηση: $Q(s, a) = V(\text{afterstate}(s, a)) = w^\top \phi(\text{afterstate}(s, a))$.

Επιπλέον, κάνουμε την παραδοχή ότι ο παίκτης με τα λευκά θα έχει θετικό σκορ αξιολόγησης και ο παίκτης με τα μαύρα θα έχει αρνητικό σκορ αξιολόγησης. Επειδή έχουμε μόνο 2 παίκτες, ισχύει ότι η χειρότερη κίνηση για τον ένα παίκτη, είναι η καλύτερη κίνηση για τον άλλο. Η απλοποιημένη διαδικασία επιλογής κίνησης σε μορφή κώδικα είναι η ακόλουθη:

```

1 best_move = None
2 best_score = None
3
4 for move in board.get_all_legal_moves():
5     with board.temporary_move(move):
6         phi = features.phi_afterstate(board)
7         score = float(weights @ phi)
8
9     if board.white_to_move:
10         choose the highest score
11     else:
12         choose the lowest score

```

3.1 Δείγματα LSPI

Προτού ξεκινήσει η εκπαίδευση LSPI, χρησιμοποιούνται πρώτα κάποια scripts για τη δημιουργία του dataset. Αυτά μετατρέπουν τις θέσεις και τις κινήσεις σε δείγματα εκπαίδευσης (training samples). Κάθε δείγμα αποθηκεύει την απαραίτητη πληροφορία για μια ενημέρωση του LSPI χρησιμοποιώντας την ακόλουθη μορφή json:

```

1 {
2   "phi": [...],
3   "r": 0.0,
4   "done": false,
5   "fen_next": "...",
6   "feature_version": "...",
7   "reward_version": "..."
8 }

```

- phi: Διάνυσμα features για το τωρινό afterstate
- r: Άμεση ανταμοιβή (reward)
- done: Δηλώνει αν η κατάσταση είναι τελική ή όχι
- fen_next: FEN της επόμενης θέσης
- feature_version: Ποιος feature extractor χρησιμοποιήθηκε
- reward_version: Ποιο σχήμα ανταμοιβών χρησιμοποιήθηκε

3.2 LSPI training objective

Το LSPI προσπαθεί να λύσει την προσεγγιστική εξίσωση Bellman $Q(s, a) = r + \gamma Q(s', \pi(s')) \Leftrightarrow w^\top \phi = r + \gamma w^\top \phi'$, όπου

- ϕ : Διάνυσμα features του τωρινό afterstate
- r : Άμεση ανταμοιβή

- γ : Discount factor
- ϕ' : Διάνυσμα features του επόμενου afterstate που επιλέχθηκε από την τωρινή άπληστη πολιτική (greedy policy)

Το LSPI μετατρέπει το παραπάνω σε ένα γραμμικό σύστημα ελαχίστων τετραγώνων (least-squares linear system): $Aw = b$, όπου το dataset αθροίζει $A = A + \phi(\phi - \gamma\phi')^\top$ και $b = b + \phi r$. Αν το δείγμα είναι τερματικό, τότε δεν υπάρχει επόμενο διάνυσμα feature, $\phi' = 0$, και η ανανέωση γίνεται $A = A + \phi\phi^\top$ και $b = b + \phi r$.

Το νέο διάνυσμα βαρών προκύπτει λύνοντας την εξίσωση $w = A^{-1}b$.

Στην πράξη, η υλοποίηση χρησιμοποιεί και έναν όρο κανονικοποίησης, οπότε προκύπτει $(A + \lambda I)w = b$, όπου λ η ισχύς κανονικοποίησης.

3.3 Training loop

Το script εκπαίδευσης εκτελεί επαναλαμβανόμενα εκτίμηση πολιτικής (policy evaluation) και βελτίωση πολιτικής (policy improvement).

Ο βρόχος εκπαίδευσης είναι ο ακόλουθος:

1. Αρχικοποίηση διανύσματος βαρών.
2. Ανάγνωση δειγμάτων εκπαίδευσης (training samples) από το dataset.
3. Για κάθε δείγμα κατασκεύασε το ταμπλό (κλάση Board) από το fen_next.
4. Χρησιμοποίηση των τωρινών βαρών για επιλογή επόμενης greedy κίνησης.
5. Εξαγωγή ϕ' για το επόμενο afterstate
6. Άθροιση A και b .
7. Λύση γραμμικού συστήματος για την έβρεση των νέων βαρών.
8. Επανάληψη μέχρις ότου σύγκλισης ή μέγιστου αριθμού επαναλήψεων.

Η σύγκλιση υπολογίζεται όταν η νόρμα της διαφοράς των νέων με των προηγούμενων βαρών είναι κάτω από μια τιμή ανοχής $\|w_{new} - w_{old}\| \leq tolerance$.

Ακολουθεί μια απλοποιημένη υλοποίηση:

```

1 w = initial_weights()
2
3 for iteration in range(max_iters):
4     A = reg * I
5     b = zeros(dim)
6
7     for sample in dataset:
8         phi = sample.phi
9         r = sample.r
10        done = sample.done
11
12        if done:
13            phi_next = zeros(dim)
14        else:
15            board = Board()
16            board.init_board(sample.fen_next)
17
18            next_move = greedy_action(board, w)
19            phi_next = features.phi_sa(board, next_move)
20
21        A += outer(phi, phi - gamma * phi_next)
22        b += phi * r
23
24    new_w = solve(A, b)
25
26    if norm(new_w - w) < tolerance:
```

```

27         break
28
29     w = new_w

```

3.4 Greedy policy improvement

Η βελτίωση πολιτικής (policy improvement) συμβαίνει έμμεσα κατά την εκπαίδευση. Για κάθε μη τερματικό δείγμα, το script κατασκευάζει την επόμενη θέση από το `fen_next` και ρωτάει: “Ποια κίνηση θα εκτελούσε η τωρινή πολιτική από την τωρινή επόμενη θέση;”. Αυτή η κίνηση προσδιορίζει το ϕ' .

Η άπληστη κίνηση (greedy action) υπολογίζεται αξιολογώντας όλες τις νόμιμες κινήσεις:

```

1 def greedy_action(board, w):
2     moves = board.get_all_legal_moves()
3
4     best_move = None
5     best_score = None
6
7     for move in moves:
8         phi = features.phi_sa(board, move)
9         score = w @ phi
10
11         if move is best for side to move:
12             best_move = move
13             best_score = score
14
15     return best_move

```

3.5 Checkpoints

Μετά από κάθε επανάληψη εκπαίδευσης, το μοντέλο αποθηκεύεται σε ένα αρχείο checkpoint με κατάληξη “.npz”. Το checkpoint αποθηκεύει κατ’ ελάχιστον: διάνυσμα βαρών, feature version, training configuration, training iteration.

Αυτά τα checkpoints φορτώνονται έπειτα από τους πράκτορες. Επιπλέον, προσφέρουν τη δυνατότητα συνέχειας της εκπαίδευσης από την τελευταία ολοκληρωμένη επανάληψη.

3.6 Βελτιστοποιήσεις απόδοσης

Για να μειωθεί ο χρόνος εκπαίδευσης χρειάστηκε να γίνουν ορισμένες βελτιστοποιήσεις.

3.6.1 Dataset streaming

Τα training samples αποθηκεύονται σε συμπιεσμένα αρχεία jsonl-gz. Αντί να φορτώνεται ολόκληρο το dataset στη μνήμη (κάτι δύσκολο καθώς η μνήμη είναι περιορισμένη και το dataset θέλουμε να είναι μεγάλο), το dataset επεξεργάζεται σταδιακά ως ένα stream.

Η διαδικασία με το streaming είναι η εξής:

1. Ανάγνωση δείγματος (sample).
2. Εξαγωγή ϕ , reward, done, `fen_next`.
3. Υπολογισμός ϕ' .
4. Άθροιση A και b .
5. Αντικατάσταση δείγματος με το επόμενο, στη ίδια θέση μνήμης.

3.6.2 Multiprocessing

Το πιο ακριβό μέρος της εκπαίδευσης LSPI είναι ο υπολογισμός του ϕ' . Για κάθε μη τερματικό δείγμα πρέπει να γίνουν τα ακόλουθα βήματα:

1. Κατασκευή της θέσης (κλάση Board) από το `fen_next`.

2. Γέννηση όλων των νόμιμων κινήσεων.
3. Αξιολόγηση υποψήφιων afterstate με τα τωρινά βάρη.
4. Επιλογή επόμενης greedy κίνησης.
5. Εξαγωγή του διανύσματος features του επόμενου afterstate.

Η διαδικασία αυτή είναι CPU-heavy και ανεξάρτητη μεταξύ δειγμάτων, οπότε μπορεί να παραλληλοποιηθεί.

Η παράλληλη έκδοση διαχωρίζει το dataset σε κομμάτια και τα στέλνει σε worker processes. Κάθε worker υπολογίζει τις μερικές συνεισφορές των A_w και b_w . Τέλος, η main process αθροίζει τους μερικούς πίνακες $A = \sum A_w$ και $b = \sum b_w$.

Με τη χρήση 8 threads, η εκπαίδευση έγινε 4.5 φορές ταχύτερη.

3.6.3 Caching

Το LSPI χρειάζεται μερικές επαναλήψεις μέχρι να συγκλίνει. Αυτό σημαίνει ότι το dataset πρέπει να διαβάζεται σε κάθε επανάληψη και η κάθε κίνηση να ξανα-υπολογίζεται. Για να αποφύγουμε τον πολλαπλό υπολογισμό των ίδιων κινήσεων, μπορούμε να χρησιμοποιήσουμε μια action-feature cache, όπου για κάθε fen_next, αποθηκεύουμε όλα τα διανύσματα features ϕ'_i της επόμενης μη τερματικής κίνησης, καθώς είναι ανεξάρτητα των βαρών και ακριβά στον υπολογισμό. Έτσι, αφού γίνει η επιλογή της greedy κίνησης k , επιλέγεται $\phi' = \phi'_k$.

Το ίδιο σκεπτικό μπορούμε να εφαρμόσουμε για features που είναι υπολογιστικά ακριβά καθώς απαιτούν υπολογισμό όλων των νόμιμων κινήσεων για μία ή και τις δύο πλευρές, όπως έλεγχος για κινήσεις που προκαλούν σαχ (check), απειλές για ματ, και άλλα.

Με τη χρήση του action-feature cache, η εκπαίδευση έγινε 5.5 φορές ταχύτερη.

3.6.4 Βελτιστοποιήσεις στον πυρήνα

Παραπάνω αναφέρθηκαν οι πιο σημαντικές βελτιστοποιήσεις στον κώδικα του lspi. Ωστόσο, η ταχύτητα εκπαίδευσης βασίζεται και στην ταχύτητα του πυρήνα. Παρακάτω παρουσιάζονται τα αποτελέσματα διάφορων βελτιστοποιήσεων του πυρήνα, οι οποίες δεν θα περιγραφούν αναλυτικά.

- v1: baseline - 497s
- v2: changed moves from enums to int bitmasks - 350s
- v3: precomputed piece properites - 322s
- v4: removed hot function calls - 309s
- v5: precomputed attack tables - 214s
- v6: refactored move generation - 212s
- v7: cached phi_sa - 166s
- v8: made _phi_afterstate O(1) instead of O(64) - 127s
- v9: move caching for greedy choice - 80s

4 Εκπαίδευση

4.1 Δεδομένα εκπαίδευσης

Τα μοντέλα LSPI εκπαιδεύτηκαν χρησιμοποιώντας ένα συνδυασμό πραγματικών παιχνιδιών σκακιού και συνθετικά δημιουργημένων δειγμάτων.

Τα δεδομένα από τα πραγματικά παιχνίδια πάρθηκαν από την ανοιχτή βάση δεδομένων του lichess (dataset Δεκεμβρίου 2025) και φιλτραρίστηκαν ώστε να διατηρηθούν μόνο παιχνίδια από παίκτες με rating μεγαλύτερο του 2000. Έτσι διασφαλίστηκε ότι οι παίκτες, και επομένως και οι κινήσεις τους, θα είναι σχετικά καλές.

Τα πραγματικά δείγματα έδωσαν ρεαλιστικές θέσεις, ανοίγματα, τακτικά μοτίβα, θέσεις middlegame και endgame. Ωστόσο, από μόνα τους δεν ήταν επαρκή. Για αυτό χρειάστηκε η χρήση συνθετικών / τεχνητών δεδομένων.

Τα συνθετικά δεδομένα που χρησιμοποιήθηκαν υπόκεινται σε 2 κατηγορίες

1. Material anchor samples: Είναι δείγματα που βοηθούν τον πράκτορα να μάθει την αξία του κάθε κομματιού. Χωρίς αυτό οι αξίες των κομματιών ήταν πολύ κοντά μεταξύ τους, πχ το πiónι (pawn) είχε αξία 1.1, ο αξιωματικός (bishop) είχε αξία 0.9 και η βασίλισσα (queen) είχε αξία 3.
2. Center anchor samples: Είναι δείγματα που βοηθούν τον πράκτορα να μάθει να προσπαθεί να επεκτείνεται στο κέντρο και να μειώσει κινήσεις με τη βασίλισσα.

4.2 Υπερπαράμετροι

Οι υπερπαράμετροι μεταξύ μοντέλων κρατήθηκαν κατά κύριο λόγο σταθερές, ώστε τα feature sets να μπορούν να συγκριθούν δίκαια.

- Discount factor $\gamma = 0.99$
- Regularization $\lambda = 1e - 1$
- Max LSPI iterations (μέγιστες επαναλήψεις LSPI) 10 (Το σύστημα συνέκλινε στις 6, οπότε υπήρχε πάντα σύγκλιση)
- Convergence tolerance (ανοχή σύγκλισης) $1e - 6$

4.3 Αρίθμηση μοντέλων

Κατά την εξέλιξη της εκπαίδευσης προέκυψαν διάφορα μοντέλα. Κάθε μοντέλο έχει μια δική του έκδοση $x.y$, όπου το x δηλώνει μοντέλα με διαφορετικά features και βάρη, ενώ το y δηλώνει μοντέλα με ίδια βάρη και features αλλά διαφορετική εκτιμήτρια (evaluator). Υπάρχουν οι παρακάτω εκτιμήτριες (evaluators):

- 0 - plain LSPI evaluator: Απλή εκτιμήτρια LSPI χωρίς καμία περαιτέρω επεξεργασία.
- .1 - tactical-safety evaluator: Εκτιμήτρια με κανόνες ασφαλείας. Προτού επιλέξει μια κίνηση από την εκτίμηση του LSPI, εφαρμόζει μερικούς τακτικούς ελέγχους για να αποφύγει προφανή λάθη, όπως κινήσεις που να χάνουν άμεσα υλικό.
- .2 - search-based evaluator: Εκτιμήτρια με ρηχή αναζήτηση. Αντί να εκτιμήσει μόνο το afterstate (δηλαδή την κατάσταση που προκύπτει μετά την εκτέλεση κάθε νόμιμης κίνησης), εκτελεί μια αναζήτηση τύπου minimax με βάθος 2 και χρησιμοποιεί την εκτιμήτρια LSPI στα φύλα του δέντρου αναζήτησης.

4.4 v1: basic

Το πρώτο μοντέλο LSPI είναι το v1. Χρησιμοποιεί ένα πολύ βασικό feature set και εκπαιδεύτηκε με 100k μόνο από κανονικά παιχνίδια. Έδειξε ότι το περιβάλλον και η εκπαίδευση δουλεύουν σωστά.

Οι κατηγορίες των features που περιλαμβάνει είναι οι εξής:

- material balance
- basic check/checkmate information
- side-to-move information
- simple positional/evaluation terms

4.5 v2: basic

Το v2 χρησιμοποιεί το ίδιο feature set με το v1 με μερικές βασικές επεκτάσεις. Εκεί που διαφέρει σημαντικά είναι η εκπαίδευση: χρησιμοποιεί δεδομένα και από πραγματικά παιχνίδια αλλά και συνθετικά δεδομένα material anchor. Συγκεκριμένα, εκπαιδεύτηκε με 900k samples πραγματικών παιχνιδιών και 100k material anchor samples.

Παρακάτω παρατίθενται αποτελέσματα από 160 (80 με λευκά και 80 με μαύρα) παιχνίδια μεταξύ του v2 με το προηγούμενο μοντέλο v1. W είναι οι νίκες, D οι ισοπαλίες και L οι ήττες. Avg plies είναι ο μέσος όρος κινήσεων μέχρι να τελειώσει το παιχνίδι. Παρατηρούμε ότι το v2 είναι ελάχιστα καλύτερο από το v1.

v2 vs v1: 12W-142D-6L score = 51.88% avg plies= 37.4

4.6 v3.2: basic - search

Με τη δημιουργία του v3 έγινε επίσης η δημιουργία των 2 νέων εκτιμητριών, tactical και search.

Το v3 χρησιμοποιεί το feature set του v2 με βασικές προσθήκες για να αποφεύγει ισοπαλίες. Εκπαιδεύτηκε με 750k samples πραγματικών παιχνιδιών και 250k material anchor samples.

Το v3.0 από μόνο του δεν ήταν ιδιαίτερα πιο δυνατό από το v2, ωστόσο το v3.1 (tactical) και το v3.2 (search) παρουσίασαν τη μεγαλύτερη αύξηση ικανότητας. Το LSPI από μόνο του δεν έχει μνήμη και δεν μπορεί να εξετάσει τις κινήσεις του αντιπάλου, οπότε υστερούσε πολύ.

v3.2 vs v2: 98W-59D-3L score = 79.69% avg plies= 46.7

4.7 v4.2: slim

Το v4 χρησιμοποιεί ένα μειωμένο feature set το οποίο αφαιρεί χαρακτηριστικά τα οποία υπήρχαν 3 φορές στο v3 (από τη μεριά των λευκών, από τη μεριά των μαύρων και διαφορά λευκών-μαύρων). Επιπλέον αφαιρεί features τα οποία είχαν πολύ χαμηλά βάρη και δεν προσέφεραν ουσιώδη πληροφορία. Εκπαιδεύτηκε με 150k samples από πραγματικά παιχνίδια και 50k material anchoring samples

Το feature set του v4 βελτίωσε την ικανότητα του μοντέλου. Το γεγονός αυτό υποδηλώνει ότι η ποιότητα των features είναι πιο σημαντική από το μέγεθος του dataset.

v4.2 vs v3.2: 60W-60D-40L score = 56.25% avg plies= 61.3

4.8 v5.2: center

Το v5 προσπάθησε να διορθώσει το πρόβλημα του κακού ελέγχου του κέντρου με την προσθήκη features. Επιπλέον προσπάθησε να διορθώσει το πρόβλημα των ανοιγμάτων - ο πράκτορας μετακινεί πολύ νωρίς και πολλές φορές τη βασίλισσα. Για την εκπαίδευση χρησιμοποιήθηκαν 650k samples από πραγματικά παιχνίδια, 250k material anchor samples και 100k center anchor samples.

Απεδείχθη ότι το v5.2 είναι πιο αδύναμο από το v4.2

v5.2 vs v4.2: 46W-55D-59L score = 45.94% avg plies= 65.8

4.9 v6.2: attack-map

Το v6 χρησιμοποιεί το feature set του v4 και προσθέτει επιπλέον attack-map features ώστε να κάνει καλύτερες επιθέσεις, να ασκεί περισσότερη πίεση και να μη μετακινεί κομμάτια σε τετράγωνα που ελέγχει ο αντίπαλος. Η εκπαίδευση έγινε με 150k samples από πραγματικά παιχνίδια και 50k material anchor samples.

Υπήρξε βελτίωση από το v5.2, αλλά η ικανότητά του είναι κοντά στο v4.2.

v6.2 vs v5.2: 55W-64D-41L score = 54.37% avg plies= 70.2

4.10 v7.2: api tactics

Το v7 χρησιμοποιεί το feature set του v6 και προσθέτει παραπάνω features που αποσκοπούν σε καλύτερες τακτικές, όπως οι νόμιμες κινήσεις που προκαλούν σαχ (check), απειλές ματ και απειλές βασίλισσας. Εκπαιδεύτηκε με 750k samples πραγματικών παιχνιδιών και 250k material anchor samples.

Υπήρξε σημαντική βελτίωση από το v6.2.

v7.2 vs v6.2: 63W-64D-33L score = 59.38% avg plies= 74.2

4.11 v8.2: tactics clean

Το v8 δεν προσθέτει νέα features αλλά αφαιρεί features που δεν συνεισφέρουν ουσιαστικά. Η προσθήκη πολλών features έχει κόστος στην ταχύτητα. Πριν γίνουν αφαιρέσεις, χρησιμοποιήθηκε η τακτική ablation σε καινούριο script, η οποία ελέγχει την ικανότητα του μοντέλου πριν και μετά την αφαίρεση συγκεκριμένων features. Επιπλέον αφαιρέθηκαν τα v6 features καθώς ήταν ακριβά χωρίς σημαντικό όφελος, καθώς και πολλά άλλα τακτικά features του v7.

Εκπαιδεύτηκε με 150k samples πραγματικών παιχνιδιών και 50k material anchor samples.

Αρχικά φάνηκε μικρή διαφορά αλλά έπειτα η ικανότητα του μοντέλου μειώθηκε σημαντικά.

v8.2 vs v7.2: 40W-58D-62L score = 43.12% avg plies= 61.0

4.12 v9.2: response tactics

Το v9 χρησιμοποιεί το feature set του v8 και προσθέτει επιπλέον features ξεχωριστά για την κάθε πλευρά (λευκά και μαύρα). Τα διαφορετικά features δίνουν λιγότερη πληροφορία και ενδεχομένως να χάνονται ορισμένες χρήσιμες λεπτομέρειες.

Για παράδειγμα, ένα διαφορετικό feature όπως “white_safe_capture_value - black_safe_capture_value” μπορεί να κρύβει σημαντικό περιεχόμενο. Τόσο όταν και οι 2 πλευρές έχουν ισχυρές τακτικές, όσο και όταν καμία πλευρά δεν έχει ισχυρές τακτικές, παράγεται η ίδια τιμή στο διαφορετικό feature.

Εκπαιδεύτηκε χρησιμοποιώντας 150k samples από πραγματικά παιχνίδια και 50k material anchor samples.

Παρατηρήθηκε σημαντική αύξηση της ικανότητας του μοντέλου.

v9.2 vs v8.2: 72W-59D-29L score = 63.44% avg plies= 70.5

4.13 v10.2: response fast

Το v9 είχε πολύ καλή επίδοση, ωστόσο ήταν αρκετά αργό. Ο σκοπός του v10 ήταν η δημιουργία ενός εξίσου ισχυρού μοντέλου που να τρέχει πιο γρήγορα. Χρησιμοποιεί μια μειωμένη έκδοση του feature set του v9. Αφαιρεί διάφορα features τα οποία είναι περιττά, αργά, ή προσφέρουν ελάχιστη πληροφορία στην εκτίμηση. Μείωσε τον αριθμό features από 80, σε 55. Εκπαιδεύτηκε με 150k samples πραγματικών παιχνιδιών και 50k material anchor samples.

Σε σύγκριση με το v9, το v10 είναι λιγότερο ισχυρό αλλά πιο γρήγορο.

v10.2 vs v9.2: 50W-48D-62L score = 46.25% avg plies= 72.0

5 Εκτέλεση του project

5.1 Προαπαιτούμενα

Απαιτείται έκδοση **Python 3.10** ή νεότερη. Πριν την εκτέλεση του project, συνιστάται η δημιουργία ενός Python virtual environment. Μετά τη δημιουργία του, χρειάζεται να ενεργοποιηθεί και να εγκατασταθούν οι απαραίτητες βιβλιοθήκες από το αρχείο “requirements.txt” (παρατίθεται παρακάτω η εντολή).

```
1 python -m venv .venv
2 source .venv/bin/activate
3 python -m pip install -r requirements.txt
```

5.2 GUI

Η εκτέλεση του GUI γίνεται με την ακόλουθη εντολή

```
1 python -m gui.game
```

5.3 Εκπαίδευση

Η εκπαίδευση ενός νέου μοντέλου LSPI αποτελείται από 3 κυρίως στάδια:

1. Δημιουργία training samples
2. Εκπαίδευση του μοντέλου LSPI
3. Επιθεώρηση και αξιολόγηση του μοντέλου

Ο απλούστερος τρόπος είναι με τη χρήση του ανατοματοποιημένου pipeline script

```
1 python -m scripts.run_lspi_experiment \  
2 --agent v10 \  
3 --pgn 150000 \  
4 --anchor 50000 \  
5 --min-elo 2000
```

το οποίο εκτελεί αυτόματα τα παρακάτω:

1. generate PGN-based samples
2. generate anchor samples
3. mix datasets
4. train LSPI
5. save a checkpoint
6. optionally evaluate the new model

Για παράδειγμα, η εκπαίδευση ενός μοντέλου v10 μπορεί να εκτελεστεί με:

```
1 python -m scripts.run_lspi_experiment \  
2 --agent v10 \  
3 --pgn 150000 \  
4 --anchor 50000 \  
5 --min-elo 2000 \  
6 --matrix-label v10_new \  
7 --matrix-kind tactical \  
8 --matrix-games 80 \  
9 --matrix-workers 12 \  
10 --matrix-chunk-size 2
```

Το μοντέλο θα αποθηκευθεί ως ένα .npz checkpoint στο μονοπάτι “data/processed/checkpoints/”.